

Cravitoo — Phase 1 Critical Fix Report

Date: Feb 2026 · Environment under test: **preview only** (production untouched as agreed)

TL;DR Verdict

Phase 1: PASS — all five reported critical issues are fixed, covered by automated tests, and verified end-to-end against the live preview environment. **Do not deploy to production until you have reviewed this report and approved.**

#	Issue	Severity	Status	Tests added
1	Logout / <code>/login?expired=1</code> redirect loop	High	✓ Fixed	2 backend + 5 frontend smoke
2	Public demo credentials on <code>/login</code>	High	✓ Fixed	1 backend + 1 frontend smoke
3	Public demo accounts mutating real DB	Critical	✓ Fixed	3 backend (incl. fail-secure)
4	<code>/auth/register</code> privilege escalation	Critical	✓ Fixed	7 backend (5 roles + edge cases)
5	Broken order lifecycle / stale orders	High	✓ Fixed	6 backend (transitions + atomic + stale)

Total: **21 new automated tests** · **327 pre-existing tests still passing** · **2 pre-existing failures unrelated to Phase 1.**

Root Cause of Each Issue

Bug 1 — Logout / session-expired redirect loop

Root cause. The global axios interceptor in `frontend/src/index.js` ran on every 401 (including the `/api/auth/me` probe fired on every page load by `AuthContext.checkAuth`).

When the refresh-token cookie was also expired, the interceptor unconditionally executed `window.location.href = "/login?expired=1"` — even when the user was on a public route (`/`, `/register`, `/privacy`, `/terms`). The login page itself then re-rendered, triggering `checkAuth` again → another `/auth/me` 401 → another redirect attempt. This was the loop you observed.

Bug 2 — Publicly displayed demo credentials

Root cause. `LoginPage.js` rendered a hard-coded credentials card under the password form whenever the password tab was active. The block was always visible, irrespective of environment.

Bug 3 — Demo accounts in production

Root cause. `routers/demo.py` exposed `POST /api/admin/demo/setup`, `POST /api/admin/demo/teardown`, and `GET /api/admin/demo/status` with `master-admin` auth as the only gate. There was no runtime environment check, so a master-admin (or anyone who escalated to that role per Bug 4) could seed or wipe the production DB. Additionally, `seed_demo_data()` in `server.py` ran on every startup and inserted demo users + companies + menu items into whatever DB the backend was connected to.

Bug 4 — Privilege escalation via `/auth/register`

Root cause. `RegisterRequest.role: str = "employee"` in `models.py` was a plain string with no validation. The handler in `routers/auth.py::register` accepted `data.role` verbatim and inserted it into `users.role`. A `curl -X POST /api/auth/register -d '{"role":"master_admin", ...}'` produced a master-admin account. The frontend dropdown was the only "enforcement", which a malicious client trivially bypasses.

Bug 5 — Order lifecycle / stale orders

Root cause. `PATCH /api/orders/{id}` accepted any `OrderStatus` enum value and wrote it directly to `orders.status` with `update_one({_id}, {$set: {status}})`. There was:

- No transition graph — `pending` → `completed` jumped the entire flow
 - No idempotency — the same call repeated kept "succeeding"
 - No expiry — orders created months ago remained mutable forever
 - No audit trail — status changes were untraceable
 - No concurrency control — two simultaneous calls could both succeed
-

Files Changed

Backend

File	Change
backend/.env	+ CRAVITOO_ENV="preview" (one new line)
backend/ env_config.py	NEW · fail-secure env helper (<code>is_production()</code> , <code>is_non_production()</code>)
backend/ order_lifecycle.py	NEW · state-machine, transition graph, <code>apply_transition()</code> , <code>expire_stale_orders()</code>
backend/models.py	<code>OrderStatus</code> enum extended with <code>expired</code> , <code>no_show</code> , <code>rejected</code> · comment added to <code>RegisterRequest.role</code>
backend/routers/ auth.py	<code>/auth/register</code> — server-side role lock + audit-log of attempts · domain check tightened to employee-only
backend/routers/ demo.py	<code>_guard_non_production()</code> on all 3 mutating endpoints · new <code>/admin/demo/enabled</code> public probe · <code>DEMO_CREDENTIALS_PUBLIC</code> (no passwords) · <code>_DEMO_PASSWORDS</code> kept internal
backend/server.py	<code>PATCH /orders/{id}</code> rewritten on top of <code>assert_transition_allowed</code> + <code>apply_transition</code> · <code>verify_pickup</code> re-routed through the state machine · <code>cancel_order</code> writes to <code>order_status_history</code> · <code>seed_demo_data()</code> skipped when <code>CRAVITOO_ENV=production</code>

Frontend

File	Change
frontend/src/index.js	Interceptor now skips redirect on public routes (<code>/</code> , <code>/login</code> , <code>/register</code> , <code>/privacy</code> , <code>/terms</code>) and never auto-refreshes for <code>/auth/me</code> · respects <code>skipAuthRedirect</code> config flag
frontend/src/context/ AuthContext.js	<code>checkAuth()</code> sets <code>skipAuthRedirect: true</code> so the silent session probe never bounces the user
frontend/src/pages/ LoginPage.js	Demo credentials box deleted

<code>frontend/src/pages/ RegisterPage.js</code>	Role <code><select></code> removed · replaced by a disclosure note · hard-codes <code>role: "employee"</code> in the submit payload
<code>frontend/src/pages/ master/DemoControl.js</code>	Calls <code>/admin/demo/enabled</code> first — renders "Demo Control disabled" banner if backend is in production mode

Tests

File	Change
<code>backend/tests/ test_phasel_critical_fixes.py</code>	NEW · 21 tests covering all 5 bugs
<code>backend/tests/ test_new_features.py</code>	3 tests updated to walk the valid state machine (the old tests asserted the broken behaviour, which is exactly what Bug 5 was about)

Tests Added & Actual Results

```
tests/test_phasel_critical_fixes.py — 21 passed
TestDemoGuard
  ✓ test_public_enabled_probe_responds
  ✓ test_status_never_returns_passwords
  ✓ test_setup_requires_master
TestRoleEscalation
  ✓ test_self_register_privileged_role_is_blocked[vendor]
  ✓ test_self_register_privileged_role_is_blocked[corporate_admin]
  ✓ test_self_register_privileged_role_is_blocked[site_admin]
  ✓ test_self_register_privileged_role_is_blocked[super_admin]
  ✓ test_self_register_privileged_role_is_blocked[master_admin]
  ✓ test_employee_self_register_path_still_works
  ✓ test_employee_role_case_and_whitespace_tolerated[EMPLOYEE]
  ✓ test_employee_role_case_and_whitespace_tolerated[ employee ]
  ✓ test_employee_role_case_and_whitespace_tolerated[Employee]
  ✓ test_employee_role_case_and_whitespace_tolerated[EMploYEE]
TestOrderLifecycle
  ✓ test_invalid_jump_pending_to_completed_rejected
  ✓ test_terminal_state_is_immutable
  ✓ test_valid_admin_chain
  ✓ test_idempotent_repeat_is_rejected
  ✓ test_stale_order_is_read_only
```

```
✓ test_status_history_records_each_transition
TestSessionExpiredHandling
✓ test_auth_me_returns_401_when_anonymous
✓ test_refresh_without_token_does_not_500
```

Full-suite regression

```
backend/tests - 327 passed, 1 skipped, 2 deselected (pre-existing, unrelated)
- test_dpdp_menu_push.py::test_me_data_delete_happy_path
  Pre-existing - uses @example.com which isn't in allowed_domains in this
  Independently confirmed by stashing my changes and re-running the same
- test_new_features.py::TestPaymentScope::test_other_user_cannot_view_paym
  Pre-existing - references /api/payments/checkout, a Stripe endpoint that
  was removed when Cravitoo switched to Razorpay. Endpoint genuinely does
  not exist in server.py.
```

Live end-to-end backend checks

```
✓ POST /auth/register role=master_admin → 403 "invitation only" + audit_
✓ POST /auth/register role=vendor → 403 "invitation only"
✓ GET /admin/demo/enabled (CRAVITOO_ENV=preview) → {demo_enabled: true}
✓ GET /admin/demo/enabled (CRAVITOO_ENV=production) → {demo_enabled: false}
✓ GET /admin/demo/enabled (CRAVITOO_ENV=banana) → {demo_enabled: false}
✓ GET /admin/demo/status (CRAVITOO_ENV=production) → 404
✓ POST /admin/demo/setup (CRAVITOO_ENV=production) → 404
✓ POST /admin/demo/teardown (CRAVITOO_ENV=production) → 404
✓ Demo status payload contains zero "password" fields
```

Live end-to-end frontend smoke

```
✓ Visit / anonymously → stays on / (no redirect to /login)
✓ Visit /register anonymously → stays on /register
✓ Visit /privacy anonymously → stays on /privacy
✓ Visit /terms anonymously → stays on /terms
✓ Visit /login → no Demo Accounts box visible
✓ Visit /register → role dropdown absent, disclosure note present
```

Environment & Database Changes

Environment variable (NEW)

```
CRAVITOO_ENV
```

Contract:

- Allowed non-prod values (case-insensitive): `development`, `preview`, `staging`
- Anything else, including blank/missing/typos: treated as `production`
- Production = demo endpoints disabled, demo seeding skipped

Where to set:

Environment	Value to set
Local dev	<code>CRAVITOO_ENV="development"</code>
Preview pod (this one)	<code>CRAVITOO_ENV="preview"</code> (already set)
Staging pod (if you create one)	<code>CRAVITOO_ENV="staging"</code>
<code>app.cravitoo.com</code> production	<code>CRAVITOO_ENV="production"</code> (or omit entirely — same effect)

Database

- **No migrations required.** The new `OrderStatus` values (`expired`, `no_show`, `rejected`) are added forward-only. Existing orders keep their current `status` string.
- **New collection:** `order_status_history` (created lazily on first write — no init step needed). Stores `{order_id, from_status, to_status, actor_id, actor_email, actor_role, created_at, details?}`.
- **New audit_log action:** `register_privileged_role_blocked` (existing `audit_log` collection — no schema change).

Deployment Instructions for Production

1. **Set the env var first.** On the production pod, add:

```
CRAVITOO_ENV=production
```

(Belt and braces — if you forget, the fail-secure default still kicks in.)

2. **Deploy the backend** (`server.py`, `routers/`, `env_config.py`, `order_lifecycle.py`, `models.py`).

3. **Deploy the frontend** (`index.js`, `AuthContext.js`, `LoginPage.js`, `RegisterPage.js`, `DemoControl.js`).

4. **Smoke test** (5 min, manual):

- Hit `app.cravitoo.com/api/admin/demo/enabled` — should return `{demo_enabled: false, environment: "production"}`
- Hit `app.cravitoo.com/api/admin/demo/status` while logged-in as master — should return `404 Not Found`
- `curl -X POST app.cravitoo.com/api/auth/register -d '{"role":"master_admin",...}'` — should return `403`
- Log in then log out — visit `/`, `/register`, `/privacy` — should not redirect to `/login?expired=1`

5. **Verify** that the production master-admin password no longer resembles the demo password (the README documents `ADMIN_PASSWORD` — change it on prod if it ever was `admin123`).

Rollback Instructions

This phase touched **9 backend files** and **5 frontend files** and added **3 new files** (`env_config.py`, `order_lifecycle.py`, `test_phase1_critical_fixes.py`). Two safe rollback paths:

Option A — Surgical (recommended):

```
git diff HEAD -- backend/ frontend/ > /tmp/phase1.patch
git apply -R /tmp/phase1.patch
```

Option B — Hard revert (use the Emergent "rollback" feature in your platform UI):

- Pick the checkpoint *immediately before* the message that started Phase 1.
- This brings the entire codebase back to that snapshot without git history rewrites.

The `order_status_history` collection and the `audit_log` rows tagged `register_privileged_role_blocked` are safe to leave in place — they're append-only and contain no PII beyond emails of *blocked* attackers.

Remaining Risks (still to address in later phases — out

of Phase 1 scope)

Risk	Severity	Recommended phase
Existing legacy orders older than 48h (if any) will become read-only the moment you deploy. Could surprise vendors mid-shift.	Low	One-time sweep recommended before deploy — see "Pre-deploy mitigation" below
Other endpoints (e.g. <code>/api/admin/cities</code> , <code>/api/onboarding/*</code>) need a similar IDOR / cross-tenant audit	High	Phase 2
Razorpay webhook signature is currently <i>not</i> verified (<code>RAZORPAY_WEBHOOK_SECRET</code> is blank)	High	Phase 2
No production CSP, HSTS, X-Frame-Options headers on FastAPI responses	Medium	Phase 2
Demo seeder file <code>seed_demo_data()</code> in <code>server.py:429</code> still inserts demo <code>@techcorp.com</code> users when <code>CRAVITOO_ENV != production</code> — this is intentional for preview, but worth cleaning up to use the same <code>_DEMO_PASSWORDS</code> pattern	Low	Phase 2
No rate-limit on <code>/auth/register</code> itself (only on <code>/auth/otp/request</code>) — a determined attacker could enumerate corporate domains	Medium	Phase 2
Frontend <code>LoginPage.js</code> pages/master/ <code>DemoControl.js</code> and other admin pages still show some hard-coded helper emails (in the <i>demo walkthrough text</i> , not the credential panel) — these are inside the master-admin-only <code>DemoControl</code> page, which is itself 404 in production, so safe	Low	Cosmetic — Phase 2

Pre-deploy mitigation for legacy orders

If production has many >48h-old non-terminal orders, run this one-off Mongo command to mark them *expired* *before* deploying — otherwise vendors might be surprised:

```
db.orders.updateMany(  
  { status: { $in: ["pending", "confirmed", "preparing", "ready"] },  
    created_at: { $lt: new Date(Date.now() - 48 * 3600 * 1000) } },
```



```
{ $set: { status: "expired", status_updated_at: new Date(),
          expired_by: "phase1_backfill" } }
};
```

Or alternatively, raise `ORDER_STALE_HOURS` in `order_lifecycle.py` to a value larger than your current oldest non-terminal order's age, then phase it down over a few days.

Phase 1 Verdict

✓ PASS

All five reported critical issues are fixed in code, validated by 21 new automated tests, and verified end-to-end against the live preview environment. No production data was touched. Two pre-existing test failures (Stripe checkout removal, missing example.com domain allowlist) are documented and unrelated to Phase 1.

Awaiting your approval to deploy to production.

— End of report —